

#Administrative File Organizer - Documentação Completa

Sobre o projeto

Este programa organiza arquivos de uma pasta de entrada, classifica cada um por tipo e move para subpastas de destino. Ao final, gera um relatório da execução.

1. Mapa do projeto

administrative-file-organizer/ | ├── docs/ | └── documentacao.md # este arquivo | ├── src/ | └── organizer/ | ├── paths.py # caminhos e criação de pastas | ├── scanner.py # leitura dos arquivos de entrada | ├── classifier.py # classificação por tipo de arquivo | ├── mover.py # movimentação e tratamento de duplicatas | ├── reporter.py # geração do relatório de execução | ├── data/ | ├── input/ # pasta de entrada dos arquivos | ├── output/ # pasta de saída organizada | ├── main.py # orquestra todo o programa | ├── README.md # descrição do projeto | └── requirements.txt # dependências do projeto

2. Fluxo do programa

1. O usuário roda o programa no terminal
 2. O argparse lê os argumentos --input e --output
 3. paths.py garante que as pastas de destino existem
 4. scanner.py lista todos os arquivos da pasta de entrada
 5. Para cada arquivo encontrado: a. classifier.py identifica o tipo pelo extension b. mover.py move o arquivo para a subpasta correta c. Se houver duplicata, adiciona timestamp no nome
 6. reporter.py gera um arquivo .txt com o resumo da execução
 7. O programa exibe no terminal quantos arquivos foram organizados
-

3. Código comentado

3.1 paths.py

importa o objeto Path da biblioteca pathlib

Path representa um caminho de arquivo ou pasta

```
from pathlib import Path
```

define a pasta raiz do projeto (onde está o main.py)

```
BASE_DIR = Path(file).resolve().parent.parent.parent
```

define a pasta padrão de entrada dos arquivos

```
DEFAULT_INPUT_DIR = BASE_DIR / "data" / "input"
```

define a pasta padrão de saída dos arquivos organizados

```
DEFAULT_OUTPUT_DIR = BASE_DIR / "data" / "output"
```

função que cria as subpastas de destino se não existirem

```
def ensure_directories_exist(output_dir):
```

```
# lista com os nomes de todas as categorias possíveis
categories = [
    "Images", "Documents", "Spreadsheets",
    "PDFs", "Videos", "Audio",
    "Code", "Archives", "Others"
]

# para cada categoria, cria a subpasta dentro de output_dir
for category in categories:
    (output_dir / category).mkdir(parents=True, exist_ok=True)
```

3.2 scanner.py

importa o objeto Path da biblioteca pathlib

```
from pathlib import Path
```

função que lista todos os arquivos da pasta de entrada

```
def list_files(input_dir):
```

```
# converte o caminho recebido para objeto Path
input_path = Path(input_dir)

# retorna uma lista com todos os arquivos encontrados
# iterdir() percorre o conteúdo da pasta
# is_file() garante que só arquivos são incluídos (não subpastas)
return [item for item in input_path.iterdir() if item.is_file()]
```

3.3 classifier.py

dicionário que mapeia cada categoria para suas extensões

```
FILE_CATEGORIES = { "Images": [".jpg", ".jpeg", ".png", ".gif", ".bmp", ".svg"], "Documents": [".doc", ".docx", ".txt", ".odt"], "Spreadsheets": [".xls", ".xlsx", ".csv", ".ods"], "PDFs": [".pdf"], "Videos": [".mp4", ".avi", ".mov", ".mkv"], "Audio": [".mp3", ".wav", ".aac", ".flac"], "Code": [".py", ".js", ".html", ".css", ".json"], "Archives": [".zip", ".rar", ".tar", ".gz"], }
```

função que recebe um arquivo e retorna sua categoria

```
def classify_file(file_path):
```

```
    # pega a extensão do arquivo em letras minúsculas
    extension = file_path.suffix.lower()

    # percorre cada categoria e suas extensões
    for category, extensions in FILE_CATEGORIES.items():

        # se a extensão estiver na lista, retorna a categoria
        if extension in extensions:
            return category

    # se nenhuma categoria foi encontrada, retorna "Others"
    return "Others"
```

3.4 mover.py

importa Path para manipulação de caminhos

```
from pathlib import Path
```

importa shutil para mover arquivos entre pastas

```
import shutil
```

importa datetime para gerar timestamps em duplicatas

```
from datetime import datetime
```

função que move um arquivo para a pasta de destino correta

```
def move_file(file_path, output_dir):
```

```
    # classifica o arquivo para saber a subpasta de destino
    from organizer.classifier import classify_file
    category = classify_file(file_path)

    # monta o caminho completo da pasta de destino
    destination_dir = output_dir / category

    # monta o caminho completo do arquivo de destino
    destination = destination_dir / file_path.name

    # verifica se já existe um arquivo com o mesmo nome
    if destination.exists():

        # gera um timestamp no formato AAAAMMDD_HHMMSS
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

        # cria um novo nome com o timestamp antes da extensão
        new_name = f"{file_path.stem}_{timestamp}{file_path.suffix}"

        # atualiza o destino com o novo nome
        destination = destination_dir / new_name

    # move o arquivo para o destino final
    shutil.move(str(file_path), str(destination))

    # retorna o caminho de destino e a categoria usada
    return destination, category
```

3.5 reporter.py

importa datetime para registrar a data e hora do relatório

```
from datetime import datetime
```

importa Path para manipulação de caminhos

```
from pathlib import Path
```

função que gera o relatório de execução em arquivo .txt

def generate_report(results, total, output_dir):

```
# captura a data e hora atual da execução
now = datetime.now()

# formata a data e hora para uso no nome do arquivo
timestamp = now.strftime("%Y%m%d_%H%M%S")

# monta o nome do arquivo de relatório com timestamp
report_name = f"report_{timestamp}.txt"

# monta o caminho completo do arquivo de relatório
report_path = Path(output_dir) / report_name

# abre o arquivo para escrita com encoding UTF-8
with open(report_path, "w", encoding="utf-8") as report_file:

    # escreve o cabeçalho do relatório
    report_file.write("Relatório de Execução\n")
    report_file.write(f>Data: {now.strftime('%d/%m/%Y %H:%M:%S')}\n")
    report_file.write(f>Total de arquivos processados: {total}\n")
    report_file.write("-" * 40 + "\n")

    # escreve uma linha para cada arquivo movido
    for destination, category in results:
        report_file.write(f"[{category}] {destination.name}\n")

# retorna o caminho do relatório gerado
return report_path
```

3.6 main.py

importa argparse para capturar argumentos do terminal

import argparse

importa Path para manipulação de caminhos

from pathlib import Path

importa as constantes de caminho padrão

```
from organizer.paths import DEFAULT_INPUT_DIR, DEFAULT_OUTPUT_DIR
```

importa a função que cria as pastas de destino

```
from organizer.paths import ensure_directories_exist
```

importa a função que lista os arquivos de entrada

```
from organizer.scanner import list_files
```

importa a função que move cada arquivo

```
from organizer.mover import move_file
```

importa a função que gera o relatório final

```
from organizer.reporter import generate_report
```

cria o objeto ArgumentParser com descrição do programa

```
parser = argparse.ArgumentParser( description="Organiza arquivos de uma pasta de entrada." )
```

define o argumento --input com valor padrão

```
parser.add_argument( "--input", type=str, default=str(DEFAULT_INPUT_DIR), help="Pasta de entrada dos arquivos" )
```

define o argumento --output com valor padrão

```
parser.add_argument( "--output", type=str, default=str(DEFAULT_OUTPUT_DIR), help="Pasta de saída dos arquivos organizados" )
```

lê os argumentos passados pelo usuário no terminal

```
args = parser.parse_args()
```

converte os valores recebidos para objetos Path

```
input_dir = Path(args.input) output_dir = Path(args.output)
```

garante que todas as subpastas de destino existem

```
ensure_directories_exist(output_dir)
```

lista todos os arquivos da pasta de entrada

```
files = list_files(input_dir)
```

conta o total de arquivos encontrados

```
total = len(files)
```

lista que vai guardar os resultados de cada movimentação

```
results = []
```

percorre cada arquivo encontrado

```
for file_path in files:
```

```
# move o arquivo e recebe o destino e a categoria
destination, category = move_file(file_path, output_dir)

# adiciona o resultado na lista
results.append((destination, category))

# exibe no terminal o arquivo processado
print(f"[{category}] {file_path.name}")
```

gera o relatório de execução

```
report_path = generate_report(results, total, output_dir)
```

exibe o resumo final no terminal

```
print(f"\n✅ {total} arquivo(s) organizado(s).") print(f"📄 Relatório gerado em: {report_path}")
```

4. Erros comuns

Erro 1: pasta de entrada não encontrada

Mensagem: FileNotFoundError: [WinError 2] The system cannot find the path specified

Causa: A pasta informada em --input não existe no sistema.

Solução: Verifique se o caminho está correto e se a pasta existe.

Erro 2: nenhum arquivo encontrado

Mensagem: ✅ 0 arquivo(s) organizado(s).

Causa: A pasta de entrada existe mas está vazia.

Solução: Adicione arquivos na pasta de entrada antes de rodar o programa.

Erro 3: módulo não encontrado

Mensagem: ModuleNotFoundError: No module named 'organizer'

Causa: O programa foi rodado fora da raiz do projeto.

Solução: Navegue até a raiz do projeto antes de rodar: `cd C:\Users\csb_b\dev\administrative-file-organizer`

Erro 4: ambiente virtual não ativado

Mensagem: O terminal não reconhece os módulos instalados.

Causa: O ambiente virtual .venv não foi ativado antes de rodar.

Solução: No terminal, na raiz do projeto, rode: `.venv\Scripts\activate`

5. Passo a passo para recriar do zero

Passo 1: criar a estrutura de pastas

No terminal, rode: `mkdir administrative-file-organizer cd administrative-file-organizer mkdir src mkdir src\organizer mkdir data mkdir data\input mkdir data\output mkdir docs`

Passo 2: criar o ambiente virtual

No terminal, na raiz do projeto, rode: `python -m venv .venv .venv\Scripts\activate`

Passo 3: criar os arquivos base

Crie os seguintes arquivos vazios:

- main.py (na raiz)
- requirements.txt (na raiz)
- src\organizer_init_.py
- src\organizer\paths.py
- src\organizer\scanner.py
- src\organizer\classifier.py
- src\organizer\mover.py
- src\organizer\reporter.py

Passo 4: ordem de desenvolvimento

Desenvolva os arquivos nesta ordem:

1. paths.py → constantes e criação de pastas
2. scanner.py → leitura dos arquivos de entrada
3. classifier.py → classificação por extensão
4. mover.py → movimentação e tratamento de duplicatas
5. reporter.py → geração do relatório
6. main.py → orquestração e argparse

Passo 5: testar o programa

Adicione arquivos de teste na pasta data\input e rode: python main.py

Para testar com pastas personalizadas, rode: python main.py --input C:\SuaPasta --output C:\Destino

Passo 6: inicializar o Git

No terminal, na raiz do projeto, rode: git init git add . git commit -m "Estrutura inicial do projeto"
